FINAL REPORT: Detection of Offensive Language in Portuguese (HateBR)

1. Project Introduction and Context

The main objective of this project is the classification of Instagram comments in Portuguese (HateBR dataset) into two classes: "offensive" (1) and "non-offensive" (0). To this end, a comparative study has been designed between three paradigms of natural language processing:

1. **Goal 1:** Rule-Based Systems (Lexicons and Patterns).
2. **Goal 2:** Classic Machine Learning (Embeddings GloVe + Traditional Classifiers).
3. **Goal 3:** Large Language Models (LLMs) run locally.

For the implementation of Goal 3, the **Ollama** (for model management) and **LangChain** (for Python orchestration) libraries have been used.

2. Design Methodology and Decisions (Goal 3)

At this stage, each "model" is defined as a specific combination of: **LLM Base + Prompt (Instruction, Few-Shot, Format) + Configuration (Temperature).**

2.1 Model Selection

Two complementary models have been selected that meet the requirements of the project (open models, transformers architecture, <8B parameters and multilingual/Portuguese capability):

1. **Call 3.2 (3B):**
    - **Justification:** As it has fewer parameters, it is a much lighter and faster model. This makes it easy to perform a high volume of queries in a short time and test different *prompt engineering* configurations efficiently during the experimental phase.
2. **Qwen 2.5 (7B-Instruct):**
    - **Justification:** By having more parameters, it is a more powerful and expressive model. It is used as the "strong" setting to compare how the quality of predictions scales by increasing the LLM's ability in a subtle task such as offense detection.

Inference Settings: A Temperature = 0.1 **was set** to prevent the model from being too creative, favoring accurate, consistent, and deterministic responses.

2.2 Prompting Strategy and Configuration Reduction

Initially, up to 6 different configurations were designed. However, preliminary tests showed that the runtime for only 20 examples was approximately 38 minutes, which was unfeasible. To optimize the process and achieve a runtime of **10-15 minutes**, the following decisions were made:

- Reduction from 6 to **4 final configurations**.
- Simpler and more direct prompt design.
- Limitation of input tokens (truncating very long comments), under the premise that the aggressive tone usually manifests itself at the beginning of the text.
- Use of a reduced test subset (75 examples).

The 4 Final Configurations evaluated are:

1. **Call 3.2:3b + Zero-shot:** No previous examples.
2. **Call 3.2:3b + Few-shots:** With 4 examples of context.
3. **Call 3.2:3b + Chain of Thought (CoT) + Four-shots:** Step-by-Step Reasoning + 4 Examples.
4. **Qwen 2.5:7b-instruct + Chain of Thought (CoT) + Four-shots:** The most powerful setup.

2.3 Data Preparation and Splitting

Data Division (Train/Val/Test): An overall proportion of **70% Train and 30% Test** has been maintained.

- **Fix in Meta 2:** In the Classic ML phase, it was detected that the validation set was being mistakenly extracted from the test set. This was corrected to establish a final split of **55% Train, 15% Validation, and 30% Test**. This ensures that Targets 1, 2, and 3 are evaluated on the same proportion of test data.
- **Goal 3:** Because it doesn't require hyperparameter tuning (as in classic ML), it wasn't necessary to use an explicit validation set.

Justification of Stratification: Although the *HateBR* dataset is perfectly balanced (3500 examples class 0 / 3500 examples class 1), it was decided to apply **stratification** in all divisions.

- *Why?* If you perform a purely random split (random_state), you run the risk of getting slightly unbalanced partitions (e.g.: 52% offensive / 48% non-offensive).
- *Benefit:* Stratification ensures an exact balance (50/50) in each partition, eliminating statistical noise and ensuring a **fair comparison** between the models of the three goals on the same type of distribution.

Evaluation Subset for Goal 3: Due to the computational and time constraints of the local LLMs, it was not possible to process the full 30% of the test set. A **stratified subset of 75 examples** (balanced between classes 0 and 1) was selected to evaluate the configurations of Target 3.

Critical Note: For this reason, the results of Target 3 are qualitative and not strictly comparable in absolute magnitude with those of Targets 1 and 2, which used the full test.

2.4 Evaluation Metrics (Goal 3)

The combination of **F1-macro + Parse_Error_Rate was used**.

- **Parse Error Rate:** Identifies the percentage of cases in which the model refuses to respond or does not return a valid tag (common in models with strict security filters).
- **Importance of False Negatives:** In content moderation, a model with high F1 but that lets offensive comments (False Negatives) pass through is dangerous. The combined metric allows us to interpret whether the model is accurate and, at the same time, if it is capable of processing toxic content without being censored.

CONCLUSIONS OF THE RESULTS (they are not exactly the same but the following is always deduced):

Que Llama3.2:3b is **overprotected** for this type of task.

- It refuses to classify comments with explicit insults → precisely the ones that interest you most.
- This is not "a bug of yours", but a **limitation of the alignment/safety** of the model.

What to add **few-shots**:

- It reduces the negative ones (parse errors go down a lot).
- But it changes the bias of the model: it becomes more likely to label as offensive.

That **Qwen2.5:7b**:

- It is less restrictive at the safety level for this academic use.
- Learn better from the few-shot.
- It offers near-perfect balance between classes → "winning" candidate.

Final Comparison and Conclusions of the Project (HateBR)

1. Comparative Table of Results

Below are the metrics obtained by the best configuration in each of the three project goals.

Goal	Approach	Best Model/Configuration	Accuracy	F1-Score (Macro)
Goal 1	**Rules**	Lexicon Hybrid + Patterns	0.839	**0.840**
Goal 2	**Classic ML**	**GloVe + SVM**	0.816	**0.816**
Goal 3	**LLMs**	**Qwen 2.5 (7B) + CoT**	0.880	**0.880***

Note: The results of Goal 3 are based on a representative stratified subset (75 examples) due to computational costs, while Goals 1 and 2 were evaluated on the full test set (30% of the dataset).

2. Comparative Analysis and Evolution

When analyzing the progression of the results, we observed how the abstraction capacity of the model influences the detection of hatred:

Goal 1 vs. Goal 2: The surprise of the explicit Contrary to what usually happens in general NLP problems, in this dataset the **Rules-Based System (0.840)** slightly outperformed the **Machine Learning model (0.816).**

- This indicates that the *HateBR* dataset contains a high density of **explicit hatred** (direct insults, taboo words). The rules capture this with surgical precision.
- The ML model (GloVe + SVM), by using averaged *embeddings*, sometimes "softens" the toxicity of a sentence. If a serious insult is surrounded by many

neutral words, the average vector may miss the offending signal, causing a false negative that the rule would not commit.

Goal 3: The semantic leap The **LLM (Qwen 2.5)** achieved the best performance **(0.880),** demonstrating that to overcome the 84-85% barrier it is necessary to understand the context and not just the words.

- The LLM with the *Chain of Thought* prompt (P4) detects sarcasm and indirect offenses in cases where the ML model fails due to lack of context and the rules fail due to lack of insults.

3. Conclusions and Error Analysis

The implementation of Goal 3 has allowed us to audit the errors of previous approaches and refine detection through *prompt engineering*:

Where does the LLM get it right best? The LLM dominates in detecting **implicit hatred and sarcasm**.

- *Example:* Comments that use complex political rhetoric or irony ("What a great favor you do the country...") are captured by the LLM thanks to their massive pre-training, while for the rules they are invisible.

Where do the rules fail the most? Rules consistently fail in two scenarios:

1. **False Negatives (Hate without insults):** Ironic phrases or subtle slights that do not contain words from the "blacklist".
2. **False Positives (Colloquial Context):** The rules system does not distinguish between an actual insult and the colloquial/affectionate use of a swear word between friends (language reappropriation), something that the LLM usually discerns better.

What kind of insults/ironies are most difficult? Non-offensive ironic expressions remain the biggest challenge. Although the LLM improves detection, it sometimes confuses harsh but legitimate political criticism with hate speech.

- *Refinement:* In Goal 3, by adding to the prompt a strict definition of *"offensive linguagem"* (differentiating it from political criticism) and using *Few-Shot* with ambiguous examples, we were able to reduce these false positives.

Biases and Model Behavior During Goal 3, we detected disparate behavior in the models evaluated:

- **Llama 3.2 (3B)** was **too strict/overprotected**. When faced with comments with strong insults, it tended to activate its security filters and refuse to respond (generating *Parse Errors*), which makes it unfeasible to classify highly toxic datasets.

- **Qwen 2.5 (7B)** turned out to be more **forgiving and analytical**, acting as an objective scorer capable of processing the offensive text without rejecting the task, which explains its superiority in the final metrics.

Final conclusion

The project concludes that while a rules system is an exceptionally strong baseline for *HateBR* due to the explicit nature of feedback, **the LLMs-based architecture (Goal 3) is the only one capable of understanding the subtleties of implicit hatred**, reaching maximum performance (0.88 F1) when properly guided with *Chain of Thought*.

BIBLIOGRAPHY:

- https://aclanthology.org/2022.lrec-1.777/ → Dataset HateBR **paper** (very important to cite it well)
- https://ollama.readthedocs.io/en/api/ → Ollama documentation/API (how models are invoked, local usage, etc.)
- https://docs.langchain.com/oss/python/integrations/providers/ollama → LangChain documentation for integration with Ollama (justify the library you use in the code)
- https://huggingface.co/meta-llama/Llama-3.2-3B)
- https://huggingface.co/Qwen/Qwen2.5-7B-Instruct